BDD2Seq: Enabling Scalable Reversible-Circuit Synthesis via Graph-to-Sequence Learning

Mingkai Miao[1], Jianheng Tang[2], Guangyu Hu[2*], Hongce Zhang[1,2*]

[1]Hong Kong University of Science and Technology (Guangzhou), Guangzhou, Guangdong, China
[2]Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong
mmiao815@connect.hkust-gz.edu.cn, {ghuae,jtangbf}@connect.ust.hk, hongcezh@ust.hk

arXiv:2511.08315v2 [cs.AR] 7 Aug 2026

Abstract

Binary Decision Diagrams (BDDs) are instrumental in many electronic design automation (EDA) tasks thanks to their compact representation of Boolean functions. In BDD-based reversible-circuit synthesis, which is critical for quantum computing, the chosen variable ordering governs the number of BDD nodes and thus the key metrics of resource consumption, such as Quantum Cost. Because finding an optimal variable ordering for BDDs is an NP-complete problem, existing heuristics often degrade as circuit complexity grows. We introduce BDD2Seq, a graph-to-sequence framework that couples a Graph Neural Network encoder with a Pointer-Network decoder and Diverse Beam Search to predict high-quality orderings. By treating the circuit netlist as a graph, BDD2Seq learns structural dependencies that conventional heuristics overlooked, yielding smaller BDDs and faster synthesis. Extensive experiments on three public benchmarks show that BDD2Seq achieves around $1.4\times$ lower Quantum Cost and $3.7\times$ faster synthesis than modern heuristic algorithms. To the best of our knowledge, this is the first work to tackle the variable-ordering problem in BDD-based reversible-circuit synthesis with a graph-based generative model and diversity-promoting decoding.

Introduction

Binary Decision Diagrams (BDDs) (Akers 1978), a compact and efficient graph representation of Boolean functions, play a significant role in electronic design automation (EDA) tasks such as logic optimization, circuit synthesis and formal verification (Hachtel and Somenzi 2005). BDDs structurally represent Boolean functions through decision nodes that branch according to Boolean variables, ultimately terminating in nodes representing constant Boolean values. Reduced Ordered Binary Decision Diagrams (ROBDDs) are among the most widely adopted BDD variants due to their canonicity, ensured by adhering to a predefined variable ordering and eliminating redundant nodes through merging isomorphic subgraphs. Consequently, for a given variable ordering, the ROBDD representation of a Boolean function is unique and compact (Bryant 1986). In the following context, we will indistinguishably use the term BDD to refer to ROBDD.

BDDs have notable applications in reversible circuit synthesis, a critical area in quantum computing and low-power design (Wille and Drechsler 2009). BDD-based synthesis methods generally provide scalability and resource efficiency advantages, especially for handling large Boolean functions (Kerntopf, Perkowski, and Podlaski 2012). Crucially, resource metrics such as Quantum Cost and gate count in reversible circuits directly correlate with the node count of their BDD representations (Soeken et al. 2016; Wille and Drechsler 2009); therefore, minimizing BDD sizes significantly improves these metrics. However, achieving compact BDD representations strongly depends on selecting an effective variable ordering; a poor ordering can dramatically increase BDD size and compromise efficiency.

As will be detailed in the next section, existing methods for deciding variable ordering often struggle to balance computational efficiency and solution quality, exhibiting inconsistent performance especially for large-scale circuits. Due to these limitations, there is significant motivation to explore alternative methods capable of learning from past experiences and structural information inherent in circuits. Recent advances in machine learning, especially techniques that leverage structural and sequential information, provide promising pathways to address these challenges. Its success in EDA tasks, ranging from circuit design optimization to hardware formal verification, demonstrates its potential to address longstanding challenges in this domain (Huang et al. 2021; Guo et al. 2023). Specifically, Graph Neural Networks (GNNs) have emerged as a powerful tool for modeling and analyzing circuit structures as they can inherently capture the relational and structural information in graphs and circuits can naturally be modeled as nodes and edges of graphs (Li et al. 2022; He et al. 2021; Zhu et al. 2022).

Since GNNs do not inherently support sequential predictions, we integrate a generative Pointer Network decoder (Vinyals, Fortunato, and Jaitly 2015), motivated by its demonstrated success in combinatorial optimization tasks (Ma et al. 2019). Unlike traditional deterministic methods, auto-regressive decoders produce probability distributions over candidate sequences. Leveraging this advantage, we employ Diverse Beam Search (Freitag and Al-Onaizan 2017; Vijayakumar et al. 2018) to simultaneously explore multiple promising candidate sequences, thus improving solution diversity and enhancing the likelihood of identifying higher-quality variable orderings.

In this work, we present a comprehensive framework

*Corresponding authors are Hongce Zhang and Guangyu Hu.

BDD2Seq for optimizing BDD variable ordering through the integration of graph-based learning and advanced decoding strategies. Our contributions include:

- **BLIF Format Circuit-to-Graph Representation**: We propose a novel method to embed circuit descriptions (in BLIF format) into graph-based representations. This circuit-to-graph embedding facilitates the application of GNN, enabling it to capture intricate structural and relational information inherent in circuit designs.
- **NLP-Inspired Sequence Generation with Diversity**. We incorporate a Pointer-Network decoder with Diverse Beam Search, which—unlike the conventional NLP beam search—retains multiple mutually dissimilar candidate orderings throughout decoding. This diversity markedly raises the chance of discovering near-optimal variable permutations in a single pass.

Extensive experiments on reversible circuit synthesis tasks validate the effectiveness of our proposed framework, demonstrating substantial and consistent improvements over traditional methods in critical metrics, particularly Quantum Cost (QC). Notably, our approach maintains significantly lower computational complexity growth as circuit size increases, addressing a key limitation of conventional heuristics.

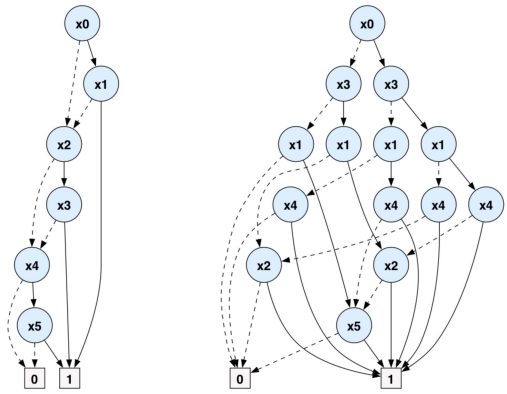
Background

Binary Decision Diagram

Binary Decision Diagrams (BDDs) are graph-based structures representing Boolean functions through decision nodes arranged according to Boolean variables. Reduced Ordered Binary Decision Diagrams (ROBDDs) refine BDDs by enforcing a fixed variable ordering and removing redundant nodes, achieving a canonical and compact representation.

The size of a BDD, indicated by the node count, significantly impacts computational efficiency and resource utilization. Consequently, optimizing BDD size is essential, as it directly correlates with performance metrics such as Quantum Cost, gate count, and computational resources in reversible synthesis tasks (Wille and Drechsler 2009; Soeken et al. 2016). One critical factor affecting BDD size is the variable ordering, which, if poorly chosen, can result in exponential growth of the BDD representation and significant slowdown of synthesis. This variability underscores the importance of selecting an effective variable ordering to maintain manageable BDD sizes. To illustrate, **Figure 1** shows the BDD representations of the same Boolean function $f = (x_0x_1 \vee x_2x_3 \vee x_4x_5)$ using two different variable orderings. The ordering $(x_0, x_1, x_2, x_3, x_4, x_5)$ yields a compact BDD structure with total node count 8, whereas the ordering $(x_0, x_3, x_1, x_4, x_2, x_5)$ leads to significant node expansion with total node count 16. This example clearly demonstrates the sensitivity of BDD node count to variable ordering and shows the importance of selecting efficient orderings to optimize performance in EDA applications.

However, identifying an optimal variable ordering is an NP-complete problem (Bollig and Wegener 1996), necessitating efficient algorithms to provide near-optimal solutions.



$ordering_1 = (x_0, x_1, x_2, x_3, x_4, x_5)$ $ordering_2 = (x_0, x_3, x_1, x_4, x_2, x_5)$

Figure 1: $f = (x_0x_1 \vee x_2x_3 \vee x_4x_5)$ with different orderings

Existing approaches typically include heuristic and exact algorithms, each with distinct characteristics. Heuristic methods, such as sifting (Rudell 1993) and its variants (e.g., symmetric sifting (Scholl et al. 1999) and group sifting (Panda and Somenzi 1995)), iteratively relocate variables to positions that locally minimize BDD size, prioritizing computational efficiency. Genetic algorithms (GAs) (Drechsler, Becker, and Göckel 1996) explore the search space more broadly by evolving candidate orderings through selection, crossover, and mutation. Linear methods (Gunther and Drechsler 1999) utilize linear transformations for node-size optimization, providing rapid results with simplicity. On the other hand, exact algorithms systematically navigate the search space using techniques like dynamic programming (Friedman and Supowit 1990; Stergiou 2011) to guarantee optimal solutions.

Despite these various approaches, significant challenges persist: heuristic methods frequently converge prematurely, missing globally optimal solutions, and suffer inconsistent performance across diverse circuits, particularly for more complex Boolean functions. Conversely, exact algorithms ensure optimal solutions but are limited by their exponential computational complexity, restricting practical use to small-scale circuits. A hybrid optimizer melding a steady-state genetic algorithm with several contemporary swarm-intelligence variants was introduced (Awad, Hawash, and Abdalhaq 2023) , translating continuous search dynamics into BDD variable permutations and delivering almost linear scalability in both size reduction and runtime. However, its reported effectiveness is validated on only seven benchmark functions, leaving the generality of those gains open to further validation. By framing Boolean functions as hypergraphs and training a 3-hypergraph model, the method proposed in (Xu et al. 2018a) predicts near-optimal variable orderings with reduced computation time; nevertheless, its BDD node-size reduction remains inferior to both the GA and the Linear approach.

To further illustrate, in **Table 1** we present a compari-

son of the BDD size and reordering time on three representative circuits from ISCAS85 (Brglez and Fujiwara 1985) and LGSynth91 (Yang 1991): `c880`, `dalu`, and `rot`, using the sifting algorithm, known for its efficiency, and the genetic algorithm, GA, typically recognized for its better performance. For `c880` and `dalu`, sifting is indeed quick but offers limited node-size reduction, while GA achieves improved reduction at the cost of significantly longer runtime. However, the results for the `rot` circuit reveal a critical issue: GA not only becomes excessively time-consuming but also severely underperforms in node-size reduction compared to sifting. This observation underscores a fundamental limitation in existing heuristic algorithms: they lack the ability to learn from the intrinsic structural properties of diverse circuits, leading to inconsistent performance.

Table 1: Sifting/GA for BDD reordering on 3 samples

Circuit	PI/PO†	Gates	Alg.‡	Time(s)	Nodes
`c880`	60/26	383	Sifting	0.74	11437
			GA	208.03	6127
`dalu`	75/16	1697	Sifting	39.48	898
			GA	63.45	772
`rot`	135/107	691	Sifting	2.73	9652
			GA	240.21	573369

† PI: *primary input*; PO: *primary output*.
‡ Alg. denotes Algorithms.

Consequently, these limitations strongly motivate developing data-driven methods capable of effectively balancing computational efficiency and consistent performance of BDD size optimization.

Reversible Circuit Synthesis

A reversible circuit is the realization of reversible logic (Toffoli 1980), where each output uniquely maps back to a specific input, ensuring that the computational process is inherently invertible. Such one-to-one mappings significantly minimize energy dissipation and power consumption, making reversible circuits particularly attractive for energy-efficient and low-power circuit designs (Landauer 1961). Quantum circuits represent a prominent subclass of reversible circuits (Chiribella, Yang, and Renner 2021). Particularly, arithmetic quantum circuits like quantum adders play a critical role as core subroutines in numerous significant quantum algorithms, such as Shor's factoring algorithm. Efficient quantum arithmetic circuits directly influence the feasibility and performance of quantum algorithms by reducing gate complexity, circuit depth, and resource demands, making their optimization crucial for practical quantum computing implementations (Gidney and Ekerå 2021).

To efficiently synthesize such arithmetic circuits and other reversible circuits derived from classical Boolean functions, prior works have proposed BDD-based approaches for reversible circuit synthesis (Wille and Drechsler 2009), which leverage the compactness and canonicity of BDDs to generate efficient reversible circuits, where each BDD node directly maps to a sequence of Toffoli and CNOT gates. Compared with alternative synthesis methods, BDD-based approaches typically achieve superior performance in critical metrics such as Gate Count and Quantum Cost (QC), which directly represent the resource requirements of quantum circuit implementations (Kerntopf, Perkowski, and Podlaski 2012).

While BDD-based reversible circuit synthesis offers a highly efficient method for generating reversible circuits, the challenge remains to be minimizing the number of BDD nodes. Fewer BDD nodes directly lead to a smaller, more efficient circuit with fewer gates (Soeken et al. 2016), which ultimately reduces Quantum Cost. **Table 2** and **Figure 7** in **Appendix** explicitly illustrate this impact by comparing reversible circuits synthesized using a natural (unoptimized) ordering with those produced by optimized variable orderings. The results clearly highlight the substantial benefits achievable through effective variable ordering optimization.

Table 2: Metrics for reversible C17 circuit with/without GA

Metric	BDD w/o Reorder	w/ GA Reorder
Gates	18	**13**
Lines	10	**9**
Quantum Costs	54	**37**
Transistor Costs	200	**144**

Graph Learning for EDA Tasks

Graph learning has recently attracted substantial interest within the field of EDA, primarily because many EDA tasks—including netlist optimization, circuit layout, and timing analysis—naturally map onto graph-based representations (Ma et al. 2020). Unlike traditional heuristic or analytical methods, graph learning explicitly leverages structural information encoded within circuit graphs, enabling models to efficiently capture relational patterns and dependencies. This capability positions graph learning methods as promising candidates for addressing complex circuit-related optimization problems, which are challenging for conventional analytical or heuristic approaches.

Pointer Network with Diverse Beam Search

A Pointer Network generates permutations by pointing to input positions, while Beam Search keeps the k best partial sequences at each step (Vinyals, Fortunato, and Jaitly 2015; Freitag and Al-Onaizan 2017). The pair already excels on permutation problems such as Traveling Salesman Problem (TSP). Because model likelihood and real-world EDA objectives (e.g., Quantum Cost) do not always coincide, we couple the decoder with Diverse Beam Search (Vijayakumar et al. 2018) to retain several mutually distinct candidates, greatly increasing the chance that at least one aligns with the downstream metric.

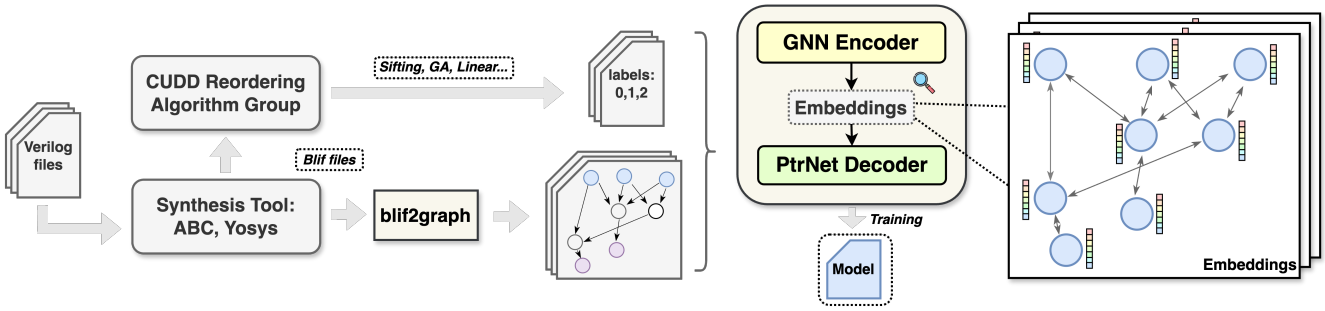


Figure 2: Overall training phase

Methodology

Overview of framework

In this section, we will describe the functionality of `BDD2Seq` in two phases. The first is the training phase, which outlines the steps of data preprocessing, label generation, graph construction, and model training. The second is the inference phase, focused on the reversible circuit synthesis task, where we apply the trained model with Diverse Beam Search strategy to generate the predicted variable ordering, which is then used to perform reversible circuit synthesis.

Training Phase. **Figure 2** overviews the overall training phase. *(i) Label generation.* Each Verilog design is first mapped to a gate-level BLIF netlist with ABC or Yosys (Brayton and Mishchenko 2010; Wolf 2016). We then run several CUDD ordering heuristics (Sifting, GA, Linear, *etc.*) (Somenzi 2009) and retain the ordering that minimises the BDD node count; this "best-known" sequence serves as the supervisory label. *(ii) Graph construction.* The netlist is converted by `BLIF2Graph` (details given later) into a directed graph whose node features include the gate truth table and a few basic topological descriptors, such as fan-in, fan-out, and level depth, providing lightweight structural context. *(iii) Model training.* A GNN encoder embeds the graph; a Pointer-Network decoder autoregressively outputs the variable order. Parameters are learned end-to-end by minimizing a weighted negative log-likelihood:

$$\mathcal{L} = \frac{1}{B} \sum_{b=1}^{B} \frac{\sum_{t=1}^{T} -\log p_\theta(y_{b,t})\, w_t\, m_{b,t}}{\sum_{t=1}^{T} m_{b,t}},$$

where w_t emphasizes early positions and $m_{b,t}$ masks padding tokens. Model parameters are optimized via backpropagation to minimize this loss.

Inference Phase with Reversible Circuit Synthesis. Once the model has been trained, the inference process begins by taking new circuit files ,followed by the same graph embedding step as described previously.

Shown as **Figure 3**, the core inference process involves leveraging the trained GNN encoder and Pointer Network decoder, combined with a Diverse Beam Search strategy, to predict high-quality variable ordering sequences. Diverse

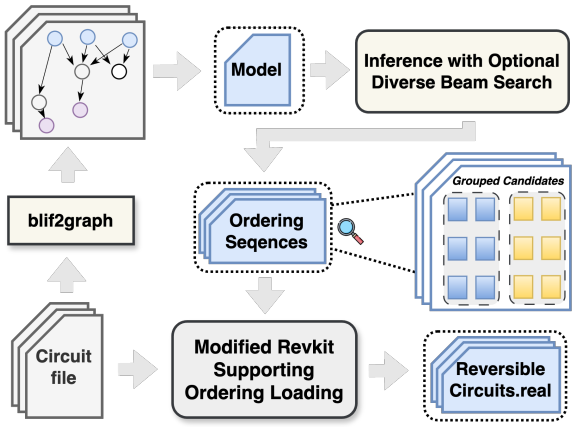


Figure 3: Inference phase with reversible circuit synthesis

Beam Search enables the model to explore multiple candidate sequences, significantly increasing the likelihood of identifying an optimal or near-optimal variable ordering.

Once generated, the predicted ordering sequence is directly provided to a modified version of the Revkit synthesis framework (Soeken et al. 2010), a widely used reversible circuit synthesis toolkit, which we adapted to accept externally provided variable orderings. With this predicted ordering, Revkit constructs the circuit BDD and then synthesizes the reversible circuit. The synthesized circuit is subsequently outputted as a *.real* file, explicitly representing the optimized reversible circuit leveraging the variable ordering predicted by our model.

Circuit Embedding - `BLIF2Graph`

Prior GNN-based work (Li et al. 2023) converts circuits into And Inverter Graph (AIG/AIGER) form before building the graph representation, a format that contains only AND gates and inverters and thus offers limited functional diversity; the lack of native support for BLIF netlists, where each gate is specified by a full truth table, motivates our richer `BLIF2Graph` embedding. As illustrated in **Figure 4**, our proposed embedding method transforms circuit descriptions provided in BLIF format into graph representations suitable for processing by GNNs. In the BLIF format, circuits

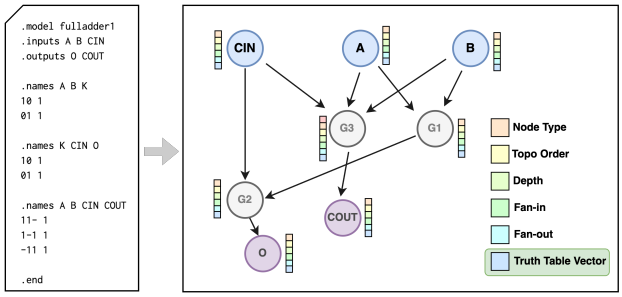


Figure 4: `BLIF2Graph`: BLIF to graph with embeddings

are defined through inputs, outputs, and logic gates (using `.names` as statement), where each logic gate's behavior is specified by a compact Boolean relationship between inputs and outputs.

A key innovation in our embedding method is the **truth-table encoding**, a scheme designed to enable the GNN to reason about the circuit's logic function, not just its topology. It compactly encodes the logical function of each gate into a uniform-length vector embedded directly as node features, allowing the GNN to accurately capture functional dependencies among nodes. **Algorithm 1** outlines this encoding process concisely.

To enrich the representation, we concatenate three lightweight structural scalars per node—topological rank, depth (longest PI-to-node path), and fan-in / fan-out—so the model can distinguish sources from sinks and recognize hubs without incurring extra message-passing steps.

Algorithm 1: Truth Table Embedding for BLIF Gates

Require: BLIF file B, predefined max vector length L
Ensure: Uniform truth-table embeddings V_g for gate g

1: **for each** gate $g \in B$ **do**
2: Extract inputs $\{x_1, x_2, ..., x_n\}$ from gate definition
3: Compute 2^n input combinations lexicographically:

$$C_g = [(0\ldots0), (0\ldots01), \ldots, (1\ldots1)]$$

4: Construct truth-table embedding vector V_g:

$$V_g[i] = \begin{cases} 1 & \text{if } C_i \text{ yields } 1, \\ 0 & \text{otherwise} \end{cases}, \quad \forall C_i \in C_g$$

5: Pad V_g to length L:

$$V_g \leftarrow [V_g; \mathbf{0}_{L-2^n}]$$

6: **end for**
7: **return** $\{V_g\}$ embeddings for all gates

Integrating these structural attributes alongside logic-based truth-table embeddings enables the GNN to simultaneously model the functional and relational complexities inherent in circuit structures, thereby facilitating effective learning for optimized BDD variable ordering. We ablate graph construction by comparing `BLIF2Graph` with prior AIG-to-Graph (Li et al. 2023) under identical settings; full details and results are in the **Appendix**.

Variable Ordering Prediction

With the circuit embeddings and GNN-based encoding presented, the next step is to decode these learned representations into variable ordering sequences. This decoding process translates graph embeddings into optimized variable permutations, directly determining the efficiency of the resulting reversible circuits.

Pointer-Network Decoding Loop. Since the variable ordering problem requires producing a permutation of the input variables, a Pointer Network (Vinyals, Fortunato, and Jaitly 2015) is a natural choice. Rather than selecting tokens from a predefined vocabulary, the decoder points to nodes in the input graph, which guarantees that every output sequence is a valid permutation. **Figure 5** sketches the decoding loop. Decoding begins with a special `<start>` token. At each step an LSTM (Hochreiter and Schmidhuber 1997) receives the previously selected variable (or the ground-truth variable provided by teacher forcing (Williams and Zipser 1989) during initial training), attends to all node embeddings, masks out nodes that have already been chosen, and converts the masked attention scores into log probabilities. The decoder samples the next variable from this distribution; during inference it may instead choose the top k candidates via Diverse Beam Search. The loop repeats until the `<end>` token is emitted, after which the collected variables constitute the final ordering passed to the synthesis backend.

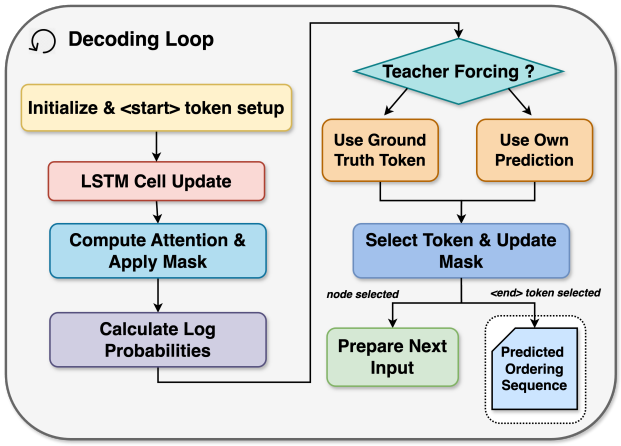


Figure 5: Pointer Network decoding loop

Diverse Beam Search. The likelihood produced by our model serves as a useful proxy, yet it is not perfectly correlated with the ultimate target, Quantum Cost (QC); relying solely on the single most-probable ordering can therefore be suboptimal and calls for a more diversified decoding strategy. To raise the chance of emitting a low-QC ordering in one forward pass, we embed *Diverse Beam Search* (Vijayakumar et al. 2018) into the decoder.

In contrast to standard beam search, Diverse Beam Search divides the beams into distinct groups, penalizing token selections repeated across these groups to encourage diversity. Formally, this diversity is enforced by applying a penalty to the raw attention scores (logits) before normalization:

$$\text{attnScores}[token] \leftarrow (1-\alpha) \times \text{attnScores}[token], \quad (1)$$

where $\alpha (0 \leq \alpha \leq 1)$ controls the intensity of diversity enforcement. The penalized scores are then normalized using a log-softmax operation to yield consistent log-probabilities.

Algorithm 2 details this algorithm. At each decoding step, candidate beams are partitioned into n groups, each containing $\frac{m}{n}$ sequences. Attention scores for each token are computed and penalized if previously selected by an earlier group within the current step. The highest-scoring tokens are appended, and the top sequences per group are retained for subsequent steps. This iterative process continues until all variables are ordered, producing diverse, high-quality variable ordering sequences.

By systematically exploring multiple diverse candidate orderings, our method enhances the likelihood of identifying high-quality variable orderings for the subsequent reversible circuit synthesis.

Algorithm 2: Diverse Beam Search for BDD Variable Ordering Prediction

Require: Beam width m, groups n, penalty α, variables X
Ensure: Set of diverse variable ordering sequences

Initialize groups $\{G_1, \ldots, G_n\}$, each with $\frac{m}{n}$ beams
for each decoding step $t = 1, 2, ..., |X|$ **do**
  for each group $G_i,\ i = 1, ..., n$ **do**
    for each candidate beam in group G_i **do**
      Compute raw attnScores for available tokens
      for token previously selected in $G_j,\ j < i$ **do**

$$\text{attnScores}[token] \leftarrow (1-\alpha) \times \text{attnScores}[token]$$

      end for
      Normalize attnScores to log probabilities:

$$\mathbf{p} \leftarrow \text{log_softmax}(\text{attnScores})$$

      Append token with highest log-probability
    end for
    Retain top $\frac{m}{n}$ sequences per group G_i
  end for
end for
return Final sequences from all groups

Experiments

Experiment Setting

Having detailed our methodology, we now proceed to empirically validate the performance of our method on BDD-based reversible circuit synthesis.

We begin by describing the dataset preparation. We construct our dataset by data augmentation techniques including circuit decomposition, random signal negation (Xu et al. 2018a), and fuzzing via `aigfuzz`. The first two techniques take circuits from the LGSynth91 benchmark (Yang 1991) as a reference. The resulted dataset comprises 5241 circuit variants, which are split into training, validation, and testing sets at a ratio of 7:2:1. As for evaluation, we take the circuits from ISCAS85 (Brglez and Fujiwara 1985), LGSynth91, and Revlib (Wille et al. 2008). Although the variants in training phase use LGSynth91 as reference, the original LGSynth91 circuits are unseen and are only used for evaluation purpose. Furthermore, Revlib and ISCAS85 are completely independent benchmarks.

All the experiments were conducted on a platform running Ubuntu 20.04.6 LTS, equipped with 2 NVIDIA GeForce RTX 3090 GPUs and dual Intel® Xeon® Platinum 8375C CPUs at 2.90 GHz.

Graph Encoder Selection

To identify an effective graph encoder for predicting BDD variable orderings, we conducted a comparative evaluation of four prominent GNN architectures: Graph Convolutional Network (GCN) (Kipf and Welling 2017), Graph Isomorphism Network (GIN) (Xu et al. 2018b), GraphSAGE (Hamilton, Ying, and Leskovec 2018), and Graph Attention Network (GAT) (Veličković et al. 2018). Ranking fidelity was measured with Kendall's τ and Spearman's ρ,

$$\tau = \frac{C-D}{\frac{1}{2}n(n-1)}, \qquad \rho = 1 - \frac{6\sum_i d_i^2}{n(n^2-1)},$$

where C and D count concordant and discordant pairs, d_i is the rank difference for element i, and n is the sequence length. As shown in **Table 6** in **Appendix**, GAT achieves the highest test-set correlation ($\tau = 0.6182$, $\rho = 0.6748$) and is thus adopted as our default encoder. Given the task difficulty, with some instances involving permutations of exceeding 200 variables, achieving these scores constitutes a strong rank correlation.

Evaluation on Reversible Circuit Synthesis

We now evaluate the practical impact on reversible circuit synthesis. Experiments are conducted on 148 combinational circuits from ISCAS85, LGSynth91, and Revlib.

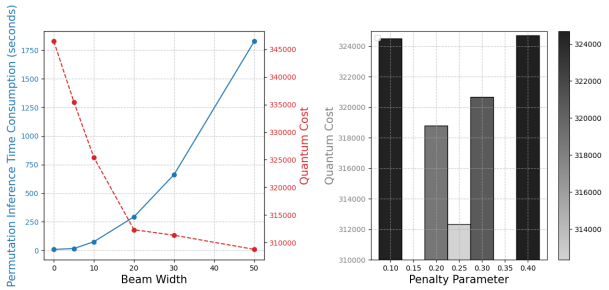


Figure 6: Ablation experiment on Diverse Beam Search

Ablation Study on Diverse Beam Search. To quantify the benefit of adapting Diverse Beam Search and to identify

Table 3: Reversible circuit synthesis metrics comparison using the evaluation dataset

Metric	Quality (50*)	Balance (20*)	Efficiency	SIFT	SYMM†	GROUP†	GA	LINEAR
Gates	**96643**	**98437**	**108181**	144112	135112	161648	2547934	134377
Lines	**28962**	**29462**	**31968**	41213	38876	45602	686839	38744
Quantum Cost	**307367**	**312345**	**346425**	471792	439452	532588	8628394	437213
Trans. Cost‡	**1164728**	**1184520**	**1309936**	1776136	1657200	2001736	32346744	1648568
Time (s)	2076.2	**521.1**	**156.2**	576.7	630.2	600.4	735.8	581.0

* The numbers in parentheses indicate beam width, i.e., Quality (50) means beam width = 50 for Quality mode.
† SYMM: symmetric sifting; GROUP: group sifting.
‡ Trans. Cost denotes the transistor cost.

Table 4: Quantum Cost comparison: `BDD2Seq` vs. traditional algorithms§

Circuit‡	PI/PO	**BDD2Seq** (QC*)		**Heuristics** (QC*)				
		Balance	**Efficiency**	**SIFT**	**SYMM†**	**GROUP†**	**GA**	**LINEAR**
dc1_142^{R}	4/7	**160**	168	160	186	186	186	181
c17^{I}	5/2	**37**	49	49	49	37	37	49
bw_116^{R}	5/28	**924**	**937**	943	937	943	937	943
con1_136^{R}	7/2	**88**	103	96	95	96	95	96
inc_170^{R}	7/9	**579**	**579**	579	592	592	592	621
alu2_96^{R}	10/6	**1218**	1415	1436	1299	1366	1298	1376
cm151a^{L}	12/2	**70**	**70**	92	92	92	92	92
add6_92^{R}	12/7	**118**	566	499	118	474	118	499
alu4_98^{R}	14/8	4455	4934	7222	**4334**	7623	4403	5583
t481_208^{R}	16/1	**139**	**140**	152	152	152	152	152
pm1^{L}	16/13	**234**	273	273	244	267	244	261
vda^{L}	17/39	4224	4224	4477	4477	4460	**4169**	4470
mux^{L}	21/1	**135**	**135**	170	170	170	170	170
cm150a_128^{R}	21/1	**136**	**136**	186	186	186	186	186
frg1_160^{R}	28/3	**598**	629	747	599	827	653	747
c880^{I}	60/26	37802	39992	61131	61131	78157	**35122**	61216
dalu^{L}	75/16	5297	5652	31145	31145	47170	**5235**	14330
x4^{L}	94/71	**2662**	**2782**	4459	4459	4334	3006	4459
apex5_104^{R}	117/88	9852	10697	10349	**9467**	10227	10283	10349
rot^{L}	135/107	**38453**	**48344**	78639	78375	110887	7339159	70456
frg2^{L}	143/139	**7189**	**7189**	12468	12361	12154	9391	12111
pair^{L}	173/137	**20754**	**20799**	46544	46444	49917	1039245	47818
Total		**135124**	**149813**	261816	256912	330317	8454773	236165

* Metric: Quantum Cost (QC).
† SYMM: symmetric sifting; GROUP: group sifting.
‡ Superscripts denote datasets: ^{I} ISCAS85, ^{L} LGSynth91, ^{R} Revlib.
§ See Table 7 and Table 8 in the **Appendix** for detailed results.

cost-effective settings, we ablate the two hyper-parameters: beam width m and diversity penalty α as shown in **Figure 6**. All other components of `BDD2Seq` are held fixed. Using $m/2$ diversity groups, increasing m from 0 to 20 cuts Quantum Cost monotonically, confirming that Diverse Beam Search indeed surfaces better orderings. Beyond m=20 the quality gain saturates while runtime rises sharply, so we adopt m=20 as the best cost-speed trade-off. With m=20, sweeping α shows a bowl-shaped curve: weak penalties (α<0.2) yield insufficient sequence variety, whereas overly strong penalties (α>0.3) scatter probability mass onto low-quality beams. The minimum Quantum Cost occurs at α=0.25, which balances exploration and exploitation. These empirically chosen values (m=20, α=0.25) define the **Balance** mode used later. The ablation confirms that Diverse Beam Search is indispensable for our performance gains and that its impact can be tuned without prohibitive runtime overhead.

Table 5: Time consumption (seconds): BDD2Seq vs. traditional algorithms§

Circuit‡	PI/PO	BDD2Seq (s)		Heuristics (s)				
		Balance	Efficiency	SIFT	SYMM†	GROUP†	GA	LINEAR
dc1_142^{R}	4/7	0.07	**0.01**	0.01	0.05	0.06	0.01	0.05
c17^{I}	5/2	0.08	**0.01**	0.07	0.05	0.06	0.01	0.01
bw_116^{R}	5/28	0.08	0.02	**0.01**	0.09	0.07	0.01	0.06
con1_136^{R}	7/2	0.13	0.02	**0.01**	0.06	0.05	0.01	0.05
inc_170^{R}	7/9	0.13	**0.01**	0.01	0.08	0.07	0.01	0.06
alu2_96^{R}	10/6	0.23	0.03	**0.01**	0.15	0.08	0.03	0.08
cm151a^{L}	12/2	0.29	0.02	0.06	0.05	0.05	0.02	**0.01**
add6_92^{R}	12/7	0.29	**0.01**	0.01	0.12	0.08	0.02	0.08
alu4_98^{R}	14/8	0.44	0.09	**0.03**	0.65	0.20	0.18	0.17
t481_208^{R}	16/1	0.47	0.03	**0.01**	0.11	0.08	0.04	0.08
pm1^{L}	16/13	0.46	0.02	0.06	0.05	0.06	0.02	**0.01**
vda^{L}	17/39	0.56	0.08	0.14	0.14	0.15	0.19	**0.03**
mux^{L}	21/1	1.03	0.39	0.24	**0.23**	0.26	0.51	0.43
cm150a_128^{R}	21/1	0.84	0.24	0.25	0.25	**0.19**	0.33	0.37
frg1_160^{R}	28/3	1.34	0.03	**0.01**	0.25	0.08	0.20	0.08
c880^{I}	60/26	26.22	**15.52**	47.71	44.43	45.20	121.45	16.86
dalu^{L}	75/16	**67.32**	**28.17**	182.95	212.63	202.55	270.63	431.41
x4^{L}	94/71	11.84	0.11	0.19	0.20	0.20	0.64	**0.04**
apex5_104^{R}	117/88	18.27	0.20	**0.08**	3.62	0.41	1.47	0.42
rot^{L}	135/107	47.58	**15.69**	61.42	60.18	61.68	139.15	20.52
frg2^{L}	143/139	27.54	0.35	1.19	1.20	1.18	2.41	**0.25**
pair^{L}	173/137	39.87	1.06	4.86	4.81	4.91	18.63	**1.03**
Total		**245.08**	**62.15**	299.32	329.39	317.67	555.94	472.03

‡ Superscripts denote datasets: ^{I} ISCAS85, ^{L} LGSynth91, ^{R} Revlib.
† SYMM: symmetric sifting; GROUP: group sifting.
§ Synthesis time reported as $0.01s$ represents Revkit's measurement limit ($\leq 0.01s$).

Overall Reversible-Synthesis Comparison. Thanks to the tunable beam width in Diverse Beam Search, BDD2Seq can be deployed in three practical modes. **Efficiency** (greedy decoding) sacrifices some optimality for minimum latency; **Balance** fixes the beam width at 20 and targets a cost–speed sweet spot; **Quality** widens the beam to 50 to chase the lowest Quantum Cost.

Table 3 shows that, regardless of modes, BDD2Seq beats classical heuristics on every metric. The Balance configuration is usually preferable: it trims Quantum Cost by $1.4\times$–$27.6\times$ while keeping runtime moderate. When turnaround time dominates, the Efficiency mode is still 3.7–$4.7\times$ faster than heuristic re-ordering yet continues to lower cost. At the other extreme, the Quality mode pushes cost even lower, albeit with proportionally longer runtime.

Detailed Evaluation on Benchmark Circuits. **Table 4** and **Table 5** contrast BDD2Seq with mainstream heuristics over 22 benchmarks spanning from 4 to 173 primary inputs, reflecting the complexity of the search space. On most of these benchmarks, BDD2Seq delivers the lowest or tied-lowest Quantum Cost. In addition, runtime grows far more slowly than that of heuristic re-ordering, making our approach both cheaper and faster as circuit complexity rises.

Between the two operating modes, **Balance** (beam width 20) secures the best or tied-best Quantum Cost on 17/22 circuits while incurring only a modest runtime increase over the ultra-fast **Efficiency** mode. In addition, although GA and Linear heuristics are reported to be two of the strongest variable-ordering techniques (Xu et al. 2018a), BDD2Seq surpasses both, delivering lower Quantum Cost and shorter synthesis time.

Conclusion

In this work, we introduced BDD2Seq, a novel deep learning approach to address the problem of optimal variable ordering in BDDs specifically for reversible circuit synthesis. From graph to sequence learning with enhanced searching strategy, our method effectively captures circuit structural and functional information and efficiently explores promising variable permutations.

Extensive experimental results confirm that BDD2Seq significantly outperforms traditional heuristic methods, achieving notable reductions in Quantum Cost and substantially faster synthesis time, particularly in circuits with large numbers of primary inputs. While its three operating modes let users trade runtime for additional cost savings when needed.

Acknowledgments

We thank Yusheng Zhao and Mingfei Yu for their helpful discussions and support for this work.

References

Akers. 1978. Binary decision diagrams. *IEEE Transactions on computers*, 100(6): 509–516.

Awad, A.; Hawash, A.; and Abdalhaq, B. 2023. A Genetic Algorithm (GA) and Swarm-Based Binary Decision Diagram (BDD) Reordering Optimizer Reinforced With Recent Operators. *IEEE Transactions on Evolutionary Computation*, 27(3): 535–549.

Bollig, B.; and Wegener, I. 1996. Improving the variable ordering of OBDDs is NP-complete. *IEEE Transactions on Computers*, 45(9): 993–1002.

Brayton, R.; and Mishchenko, A. 2010. ABC: an academic industrial-strength verification tool. In *Proceedings of the 22nd International Conference on Computer Aided Verification*, CAV'10, 24–40. Berlin, Heidelberg: Springer-Verlag. ISBN 364214294X.

Brglez, F.; and Fujiwara, H. 1985. A neutral netlist of 10 combinational benchmark circuits and a target translator. In *Fortran. ISCAS'85*.

Bryant, R. E. 1986. Graph-based algorithms for boolean function manipulation. *Computers, IEEE Transactions on*, 100(8): 677–691.

Chiribella, G.; Yang, Y.; and Renner, R. 2021. Fundamental energy requirement of reversible quantum operations. *Physical Review X*, 11(2): 021014.

Drechsler, R.; Becker, B.; and Göckel, N. 1996. Genetic algorithm for variable ordering of OBDDs. *IEE Proceedings-Computers and Digital Techniques*, 143(6): 364–368.

Freitag, M.; and Al-Onaizan, Y. 2017. Beam search strategies for neural machine translation. *arXiv preprint arXiv:1702.01806*.

Friedman, S.; and Supowit, K. 1990. Finding the optimal variable ordering for binary decision diagrams. *IEEE Transactions on Computers*, 39(5): 710–713.

Gidney, C.; and Ekerå, M. 2021. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5: 433.

Gunther, W.; and Drechsler, R. 1999. Minimization of BDDs using linear transformations based on evolutionary techniques. In *1999 IEEE International Symposium on Circuits and Systems (ISCAS)*, volume 1, 387–390 vol.1.

Guo, W.; Zhen, H.-L.; Li, X.; Luo, W.; Yuan, M.; Jin, Y.; and Yan, J. 2023. Machine learning methods in solving the boolean satisfiability problem. *Machine Intelligence Research*, 20(5): 640–655.

Hachtel, G. D.; and Somenzi, F. 2005. *Logic synthesis and verification algorithms*. Springer Science & Business Media.

Hamilton, W. L.; Ying, R.; and Leskovec, J. 2018. Inductive Representation Learning on Large Graphs. arXiv:1706.02216.

He, Z.; Wang, Z.; Bai, C.; Yang, H.; and Yu, B. 2021. Graph learning-based arithmetic block identification. In *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 1–8. IEEE.

Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation*, 9: 1735–1780.

Huang, G.; Hu, J.; He, Y.; Liu, J.; Ma, M.; Shen, Z.; Wu, J.; Xu, Y.; Zhang, H.; Zhong, K.; et al. 2021. Machine learning for electronic design automation: A survey. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 26(5): 1–46.

Kerntopf, P.; Perkowski, M.; and Podlaski, K. 2012. Synthesis of reversible circuits: A view on the state-of-the-art. In *2012 12th IEEE International Conference on Nanotechnology (IEEE-NANO)*, 1–6.

Kipf, T. N.; and Welling, M. 2017. Semi-Supervised Classification with Graph Convolutional Networks. arXiv:1609.02907.

Landauer, R. 1961. Irreversibility and Heat Generation in the Computing Process. *IBM Journal of Research and Development*, 5(3): 183–191.

Li, M.; Khan, S.; Shi, Z.; Wang, N.; Yu, H.; and Xu, Q. 2022. Deepgate: Learning neural representations of logic gates. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 667–672.

Li, Y.; Liu, M.; Mishchenko, A.; and Yu, C. 2023. Verilog-to-PyG – A Framework for Graph Learning and Augmentation on RTL Designs. arXiv:2311.05722.

Ma, Q.; Ge, S.; He, D.; Thaker, D.; and Drori, I. 2019. Combinatorial optimization by graph pointer networks and hierarchical reinforcement learning. *arXiv preprint arXiv:1911.04936*.

Ma, Y.; He, Z.; Li, W.; Zhang, L.; and Yu, B. 2020. Understanding graphs in EDA: From shallow to deep learning. In *Proceedings of the 2020 international symposium on physical design*, 119–126.

Panda, S.; and Somenzi, F. 1995. Who are the variables in your neighborhood. In *Proceedings of the 1995 IEEE/ACM International Conference on Computer-Aided Design*, ICCAD '95, 74–77. USA: IEEE Computer Society. ISBN 0818672137.

Rudell, R. 1993. Dynamic variable ordering for ordered binary decision diagrams. In *Proceedings of 1993 International Conference on Computer Aided Design (ICCAD)*, 42–47.

Scholl, C.; Moller, D.; Molitor, P.; and Drechsler, R. 1999. BDD minimization using symmetries. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 18(2): 81–100.

Soeken, M.; Frehse, S.; Wille, R.; and Drechsler, R. 2010. RevKit: A Toolkit for Reversible Circuit Design. volume 18.

Soeken, M.; Tague, L.; Dueck, G. W.; and Drechsler, R. 2016. Ancilla-free synthesis of large reversible functions using binary decision diagrams. *Journal of Symbolic Computation*, 73: 1–26.

Somenzi, F. 2009. CUDD: CU decision diagram package-release 2.4. 0. *University of Colorado at Boulder*, 21.

Stergiou, S. 2011. Implicit permutation enumeration networks and binary decision diagrams reordering. In *Proceedings of the 48th Design Automation Conference*, DAC '11, 615–620. New York, NY, USA: Association for Computing Machinery. ISBN 9781450306362.

Toffoli, T. 1980. Reversible computing. In *International colloquium on automata, languages, and programming*, 632–644. Springer.

Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. arXiv:1710.10903.

Vijayakumar, A. K.; Cogswell, M.; Selvaraju, R. R.; Sun, Q.; Lee, S.; Crandall, D.; and Batra, D. 2018. Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models. arXiv:1610.02424.

Vinyals, O.; Fortunato, M.; and Jaitly, N. 2015. Pointer networks. *Advances in neural information processing systems*, 28.

Wille, R.; and Drechsler, R. 2009. BDD-based synthesis of reversible logic for large functions. In *2009 46th ACM/IEEE Design Automation Conference*, 270–275.

Wille, R.; Große, D.; Teuber, L.; Dueck, G. W.; and Drechsler, R. 2008. RevLib: An Online Resource for Reversible Functions and Reversible Circuits. In *38th International Symposium on Multiple Valued Logic (ismvl 2008)*, 220–225.

Williams, R. J.; and Zipser, D. 1989. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2): 270–280.

Wolf, C. 2016. Yosys open synthesis suite.

Xu, F.; He, F.; Xie, E.; and Li, L. 2018a. Fast OBDD reordering using neural message passing on hypergraph. *arXiv preprint arXiv:1811.02178*.

Xu, K.; Hu, W.; Leskovec, J.; and Jegelka, S. 2018b. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*.

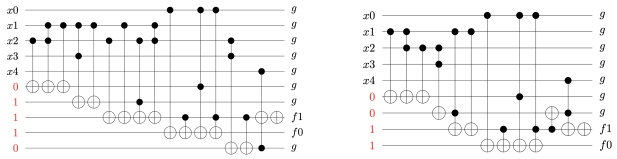
Yang, S. 1991. *Logic synthesis and optimization benchmarks user guide: version 3.0*. Citeseer.

Zhu, K.; Chen, H.; Turner, W. J.; Kokai, G. F.; Wei, P.-H.; Pan, D. Z.; and Ren, H. 2022. Tag: Learning circuit spatial embedding from layouts. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, 1–9.

Appendix

Schematics of Reversible C17 Circuit with or without BDD Variable Reordering

Figure 7 visually compares the reversible implementation of the C17 benchmark under two conditions: (a) without BDD variable-ordering optimization and (b) with an optimized ordering obtained by the genetic algorithm. The side-by-side schematics highlight how an appropriate reordering of decision-diagram variables translates into a noticeably more compact reversible realization, with shorter cascades of Toffoli controls and fewer ancillary interactions.



(a) Without Reordering (b) With Reordering

Figure 7: Schematics for reversible C17 circuit w/wo GA

GNN Encoder Selection and Training Hyper-parameters

To isolate the effect of the message-passing scheme itself, we evaluated four standard GNN encoders, namely, GCN, GIN, GraphSAGE, and GAT under identical architectural settings (hidden dimension 512, 6 layers). As summarised in **Table 6**, GAT attains the highest average Kendall's τ (0.6182) and Spearman's ρ (0.6748); hence it is adopted as the default encoder in all subsequent experiments.

- **GNN hidden dimension:** 512
- **Number of GNN layers:** 6
- **Batch size:** 16
- **Epochs:** 400
- **Optimizer:** Adam
- **Learning rate:** 1×10^{-5}
- **Random seed:** 42

Table 6: Performance comparison for GNN encoders

Encoder	Average Kendall's Tau	Average Spearman's Rho
GCN	0.4540	0.5216
GIN	0.5129	0.5877
GraphSage	0.5978	0.6589
GAT	**0.6182**	**0.6748**

Comparison on BDD Nodes and Time Consumption

Having selected **GAT** as our encoder, we next evaluate our model's performance in terms of scalability and compression effectiveness compared with traditional heuristic algorithms on LGSynth91 benchmark. The comparative results are illustrated in **Figure 8**, highlighting two key performance dimensions: time efficiency and node size reduction performance.

In terms of the time efficiency of reordering, the left side of **Figure 8** compares the reordering time required by each algorithm relative to the original BDD node sizes before reordering. Quadratic polynomials are fitted to the data points in a log-log scale to depict trends. Our approach consistently exhibits shorter and more predictable runtimes

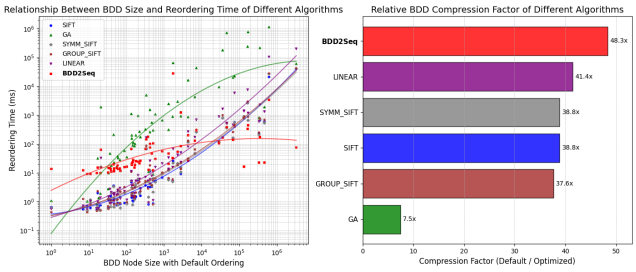


Figure 8: Time consumption and node size comparison

across varying node sizes compared to heuristic-based algorithms, whose runtimes grow substantially with node sizes. Heuristic-based algorithms show a rapid escalation in computation time as the circuit complexity grows, while our method maintains a relatively steady and efficient scaling trend, demonstrating better scalability.

On the right side of the **Figure 8**, we quantify the performance of each algorithm through the relative BDD compression factor, defined as the ratio between original BDD node counts and optimized BDD node counts. Our method achieves the highest compression ratio $(48.3\times)$, outperforming all other traditional approaches. This result highlights the effectiveness of our model, indicating superior capability in learning optimal or near-optimal variable ordering compared to heuristic methods.

Ablation study on `BLIF2Graph` and `AIG2Graph` on BDD variable ordering task

To isolate the effect of the circuit–to–graph representation, we keep the graph encoder fixed as a GAT and hold all training/configuration settings constant (optimizer, budget, decoder, batching). The only change is the graph construction: our `BLIF2Graph` versus an `AIG2Graph` obtained by `strash` then edgelist extraction. On ISCAS85, AIG's 2-input normalization inflates graphs: average node and edge counts are $2.40\times$ and $1.39\times$ those of `BLIF2Graph`, respectively. Correspondingly, end-to-end wall-clock (preprocessing + GNN inference) is reduced by 32.9% with `BLIF2Graph`.

Despite being more compact, `BLIF2Graph` also improves order quality on the held-out test set: Kendall's $\tau = 0.6182$ and Spearman's $\rho = 0.6748$, versus $\tau = 0.6038$ and $\rho = 0.6517$ for `AIG2Graph`. For BDD variable ordering, preserving gate-functional semantics at the BLIF level yields smaller graphs, lower wall-clock, and better ranking accuracy than first reducing to AIG.

Complete Comparison Results for Table 4

Next two pages present the table for all successfully synthesized reversible circuits in evaluation using different algorithms. Due to its length, we have to split it into two separate tables across two pages.

Table 7: Quantum Cost and time consumption: BDD2Seq vs. traditional algorithms (Part 1)

Circuit	BDD2Seq(B*)		BDD2Seq(E*)		SIFT		SYMM_SIFT		GROUP_SIFT		GA		LINEAR	
	QC†	Time(s)‡	QC	Time(s)	QC	Time(s)	QC	Time(s)	QC	Time(s)	QC	Time(s)	QC	Time(s)
4gt10_22.pla	17	1.22	17	2.13	17	0.01	17	0.07	23	0.05	17	0.01	17	0.05
4gt11_23.pla	5	0.07	5	0.01	5	0.01	5	0.04	5	0.06	5	0.01	5	0.06
4gt12_24.pla	17	0.06	17	0.01	17	0.01	17	0.05	16	0.05	17	0.01	17	0.05
4gt13_25.pla	10	0.07	10	0.01	10	0.01	10	0.05	10	0.05	10	0.01	10	0.05
4gt4_20.pla	19	0.06	25	0.01	19	0.01	19	0.06	23	0.06	19	0.01	19	0.05
4gt5_21.pla	12	0.07	12	0.01	12	0.01	12	0.06	18	0.05	12	0.01	12	0.05
4mod5_8.pla	24	0.07	24	0.02	24	0.01	24	0.05	24	0.06	24	0.01	7	0.05
4mod7_26.pla	86	0.06	86	0.01	86	0.01	86	0.06	80	0.05	86	0.01	64	0.05
5xp1_90.pla	254	0.13	280	0.01	254	0.01	254	0.07	254	0.06	280	0.01	254	0.06
9symml_91.pla	206	0.2	206	0.03	206	0.01	206	0.07	206	0.06	206	0.01	206	0.06
C17_117.pla	37	0.08	37	0.01	49	0.01	37	0.06	37	0.05	37	0.01	49	0.05
C7552_119.pla	202	0.08	202	0.01	202	0.01	202	0.06	202	0.05	202	0.01	202	0.06
add6_92.pla	118	0.29	566	0.01	499	0.01	118	0.11	474	0.08	118	0.02	499	0.08
adr4_93.pla	74	0.16	74	0.02	74	0.01	74	0.06	74	0.05	74	0.01	47	0.06
alu1_94.pla	139	0.29	139	0.02	139	0.01	139	0.07	139	0.09	139	0.01	139	0.05
alu2_96.pla	1218	0.23	1415	0.03	1436	0.01	1299	0.15	1366	0.08	1298	0.03	1376	0.08
alu3_97.pla	430	0.22	464	0.01	644	0.01	430	0.1	497	0.06	430	0.02	644	0.06
alu4_98.pla	4455	0.44	4934	0.09	7222	0.03	4334	0.65	7623	0.2	4403	0.18	5583	0.17
alu_9.pla	29	0.08	29	0.02	29	0.01	29	0.06	35	0.05	29	0.01	29	0.05
apex2_101.pla	3103	2.37	3178	0.16	5922	0.14	2714	1.57	6439	0.36	2939	2.78	5922	0.35
apex4_103.pla	8409	0.42	8813	0.19	8343	0.02	8236	0.55	8381	0.16	8231	0.07	8336	0.16
apex5_104.pla	9852	18.27	10697	0.2	10349	0.08	9467	3.62	10227	0.41	10283	1.47	10349	0.42
apex6_orig.pla	3895	23.86	4021	0.16	4827	0.26	4815	0.26	4274	0.26	4046	0.7	4929	0.06
apex7_orig.pla	1455	3.41	1456	0.07	2652	0.14	2652	0.13	2377	0.14	1442	0.48	2215	0.02
apla_107.pla	732	0.22	955	0.03	1002	0.01	737	0.11	979	0.07	731	0.01	979	0.06
b1_orig.pla	22	0.05	22	0.01	22	0.05	22	0.05	22	0.05	22	0.01	10	0.01
b9_orig.pla	737	2.43	737	0.05	795	0.09	801	0.08	831	0.09	720	0.2	814	0.01
bw_116.pla	924	0.08	937	0.02	943	0.01	937	0.09	943	0.07	937	0.01	943	0.06
c17.pla	37	0.08	49	0.01	49	0.07	49	0.05	37	0.06	37	0.01	49	0.01
c432.pla	12255	11.39	20391	6.79	11101	7.0	11101	6.73	11141	6.89	10977	9.14	11101	1.37
c880.pla	37802	26.22	39992	15.52	61131	47.71	61131	44.43	78157	45.2	35122	121.45	61216	16.86
c8_orig.pla	432	1.21	436	0.04	445	0.06	445	0.07	486	0.07	444	0.12	445	0.01
cc_orig.pla	272	0.72	278	0.03	379	0.06	379	0.06	379	0.07	287	0.03	379	0.01
cht_orig.pla	714	3.15	714	0.06	714	0.07	714	0.07	714	0.07	714	0.09	714	0.01
clip_124.pla	695	0.18	698	0.02	704	0.01	526	0.1	520	0.07	512	0.01	419	0.07
cm138a_orig.pla	104	0.11	104	0.01	104	0.05	104	0.06	104	0.05	104	0.01	104	0.01
cm150a_128.pla	136	0.84	136	0.24	186	0.25	186	0.25	186	0.19	186	0.33	186	0.32
cm150a_orig.pla	136	0.93	136	0.34	186	0.18	186	0.18	186	0.21	186	0.3	186	0.37
cm151a_129.pla	298	0.61	298	0.02	298	0.01	298	0.11	298	0.06	298	0.05	298	0.06
cm151a_orig.pla	70	0.29	70	0.02	92	0.06	92	0.05	92	0.05	92	0.02	92	0.01
cm152a_130.pla	62	0.25	62	0.02	62	0.01	62	0.07	62	0.06	62	0.02	62	0.05
cm152a_orig.pla	62	0.25	81	0.02	62	0.06	62	0.05	62	0.06	62	0.01	62	0.01
cm162a_orig.pla	222	0.36	222	0.03	192	0.05	247	0.05	219	0.06	219	0.03	192	0.01
cm163a_133.pla	244	0.45	267	0.02	273	0.01	267	0.08	267	0.06	244	0.02	261	0.07
cm163a_orig.pla	136	0.45	136	0.03	124	0.06	124	0.05	137	0.06	124	0.02	203	0.01
cm42a_125.pla	117	0.07	117	0.01	117	0.01	117	0.05	117	0.06	117	0.01	117	0.05
cm42a_orig.pla	117	0.05	117	0.02	117	0.05	117	0.06	117	0.06	117	0.01	117	0.01
cm82a_126.pla	44	0.07	44	0.01	82	0.01	44	0.05	44	0.04	44	0.01	49	0.05
cm82a_orig.pla	44	0.07	44	0.01	82	0.05	44	0.05	44	0.06	44	0.01	49	0.01
cm85a_127.pla	227	0.25	283	0.03	275	0.01	218	0.08	221	0.05	207	0.02	127	0.06
cm85a_orig.pla	225	0.25	225	0.02	275	0.05	285	0.06	221	0.06	207	0.01	127	0.01
cmb_134.pla	153	0.45	153	0.02	153	0.01	153	0.09	153	0.09	153	0.03	153	0.06
cmb_orig.pla	153	0.45	153	0.03	153	0.06	153	0.05	153	0.05	153	0.03	153	0.01
co14_135.pla	159	0.36	159	0.02	159	0.01	159	0.08	159	0.06	159	0.03	159	0.06
comp_orig.pla	824	45.14	824	28.75	961	92.68	1071	87.08	884	92.8	824	69.75	436	35.81
con1_136.pla	88	0.13	103	0.03	96	0.01	95	0.06	96	0.05	95	0.01	96	0.05
cordic_138.pla	344	0.87	446	0.05	325	0.03	325	0.25	315	0.15	306	0.14	220	0.14
cordic_orig.pla	320	0.86	320	0.06	325	0.15	323	0.14	315	0.16	306	0.15	220	0.03
count_orig.pla	441	1.8	441	0.04	441	0.08	441	0.08	441	0.09	441	0.2	441	0.01
cu_141.pla	219	0.36	230	0.02	220	0.01	220	0.09	224	0.06	224	0.03	234	0.06
cu_orig.pla	224	0.36	231	0.02	220	0.06	220	0.06	224	0.06	224	0.02	234	0.01
dalu_orig.pla	5297	67.32	5652	28.17	31145	182.95	31145	212.63	47170	202.55	5235	270.63	14330	431.41
dc1_142.pla	160	0.07	168	0.01	160	0.01	186	0.05	186	0.06	186	0.01	181	0.05
dc2_143.pla	431	0.15	431	0.03	431	0.01	431	0.07	431	0.06	431	0.01	431	0.05
decod24-enable_32.pla	38	0.05	38	0.01	38	0.01	38	0.04	38	0.06	38	0.01	38	0.06
decod_137.pla	202	0.08	202	0.01	202	0.01	202	0.07	202	0.06	202	0.01	202	0.05
decod_orig.pla	202	0.07	202	0.01	202	0.06	202	0.06	202	0.06	202	0.01	202	0.01
dist_144.pla	975	0.17	979	0.04	975	0.01	975	0.12	979	0.08	979	0.02	968	0.08
dk17_145.pla	429	0.21	429	0.03	426	0.01	426	0.09	574	0.07	426	0.01	426	0.06
dk27_146.pla	131	0.18	131	0.02	144	0.01	144	0.06	150	0.05	140	0.01	144	0.06
e64_149.pla	1019	5.9	1208	0.17	907	0.02	886	1.8	1124	0.11	886	0.98	907	0.1

* The letter in parentheses indicates synthesis mode, i.e., (B) means **Balance** mode, (E) means **Efficiency** mode.

†Quantum Cost

‡Synthesis times reported as $0.01s$ represent Revkit's measurement limit ($\leq 0.01s$).

Table 8: Quantum Cost and time consumption: BDD2Seq vs. traditional algorithms (Part 2)

Circuit	BDD2Seq(B*)		BDD2Seq(E*)		SIFT		SYMM_SIFT		GROUP_SIFT		GA		LINEAR	
	QC†	Time(s)‡	QC	Time(s)	QC	Time(s)	QC	Time(s)	QC	Time(s)	QC	Time(s)	QC	Time(s)
ex1010_155.pla	9633	0.41	9810	0.14	9766	0.03	9824	0.63	9696	0.21	9792	0.19	9732	0.21
ex1_150.pla	8	0.1	8	0.01	8	0.01	8	0.05	8	0.05	8	0.01	8	0.04
ex2_151.pla	60	0.09	68	0.01	73	0.01	70	0.05	60	0.06	60	0.01	49	0.05
ex3_152.pla	38	0.1	38	0.02	61	0.01	48	0.05	43	0.05	48	0.01	37	0.05
ex5p_154.pla	1837	0.22	1871	0.06	1843	0.01	1843	0.17	1843	0.11	1843	0.02	1843	0.1
example2_156.pla	1218	0.24	1415	0.03	1436	0.01	1299	0.15	1366	0.08	1298	0.02	1376	0.08
example2_orig.pla	1800	9.74	1806	0.08	2085	0.14	2085	0.15	2063	0.15	1791	0.53	2000	0.03
f2_158.pla	108	0.07	113	0.01	113	0.01	113	0.05	113	0.05	108	0.01	100	0.05
f51m_159.pla	2427	0.6	2517	0.14	5392	0.04	2433	0.49	4603	0.24	2357	0.16	4825	0.2
frg1_160.pla	598	1.34	629	0.03	747	0.01	599	0.25	827	0.08	653	0.2	747	0.08
frg1_orig.pla	757	1.35	762	0.03	747	0.07	747	0.07	827	0.07	653	0.2	747	0.01
frg2_161.pla	8584	26.96	10768	0.37	12468	0.25	7313	6.66	12154	1.36	9391	2.39	12111	1.34
frg2_orig.pla	7189	27.54	7189	0.35	12468	1.19	12361	1.2	12154	1.18	9391	2.41	12111	0.25
in0_162.pla	2296	0.42	2417	0.04	2283	0.02	2327	0.3	2328	0.09	2303	0.09	2249	0.09
inc_170.pla	579	0.13	579	0.01	579	0.01	592	0.08	592	0.07	592	0.01	621	0.06
k2_orig.pla	10432	3.33	10686	0.19	11058	0.29	11058	0.29	11302	0.31	10304	2.08	10894	0.11
lal_orig.pla	478	1.07	496	0.04	527	0.06	774	0.06	724	0.07	506	0.08	464	0.01
life_175.pla	204	0.19	210	0.01	204	0.01	204	0.09	204	0.08	210	0.02	179	0.07
majority_176.pla	41	0.08	41	0.01	41	0.01	41	0.05	41	0.05	41	0.01	41	0.05
majority_orig.pla	41	0.07	41	0.01	41	0.05	41	0.06	41	0.05	41	0.01	41	0.01
max46_177.pla	562	0.19	596	0.02	598	0.01	561	0.08	598	0.06	561	0.01	598	0.06
misex1_178.pla	281	0.16	289	0.02	304	0.01	279	0.07	302	0.06	287	0.01	304	0.06
misex3_180.pla	3830	0.44	4054	0.09	4661	0.04	3795	0.6	4619	0.24	3789	0.21	4657	0.24
misex3c_181.pla	3969	0.4	4457	0.05	4769	0.02	4165	0.57	4823	0.14	3945	0.23	4725	0.14
mlp4_184.pla	1191	0.17	1228	0.03	1158	0.01	1158	0.11	1158	0.07	1159	0.02	1158	0.06
mux_185.pla	135	1.18	224	0.26	170	0.59	170	0.34	170	0.28	170	0.63	170	0.39
mux_orig.pla	135	1.03	135	0.4	170	0.24	170	0.23	170	0.26	170	0.51	170	0.43
my_adder_orig.pla	352	64.22	352	48.54	352	169.32	352	171.76	352	165.44	352	74.96	208	58.36
one-two-three_27.pla	44	0.05	44	0.01	44	0.01	44	0.05	44	0.05	44	0.01	44	0.05
pair_orig.pla	20754	39.87	20799	1.06	46544	4.86	46444	4.81	49917	4.91	1039245	18.63	47818	1.03
parity_188.pla	31	0.99	31	0.44	31	0.55	31	2.08	31	2.02	31	0.59	31	2.01
parity_orig.pla	31	0.92	31	0.61	31	2.1	31	2.07	31	2.13	31	0.62	31	0.55
pcle_orig.pla	298	0.61	298	0.02	298	0.06	310	0.05	298	0.05	298	0.05	298	0.01
pcler8_190.pla	124	0.46	137	0.01	124	0.01	124	0.08	137	0.06	124	0.02	203	0.06
pcler8_orig.pla	765	1.13	765	0.02	639	0.06	639	0.06	639	0.07	639	0.13	639	0.01
pdc_191.pla	6742	0.72	6742	0.23	6500	0.09	6599	1.19	6500	0.47	6599	0.39	6098	0.42
pm1_192.pla	117	0.06	117	0.02	117	0.01	117	0.06	117	0.06	117	0.01	117	0.06
pm1_orig.pla	234	0.46	273	0.03	273	0.06	244	0.05	267	0.06	244	0.02	261	0.01
radd_193.pla	74	0.16	140	0.01	217	0.01	74	0.07	212	0.07	74	0.01	217	0.06
rd53_68.pla	98	0.07	98	0.01	98	0.01	98	0.06	98	0.05	98	0.01	65	0.07
rd73_69.pla	217	0.12	217	0.02	217	0.01	217	0.06	217	0.06	217	0.01	148	0.06
rd84_70.pla	304	0.17	304	0.02	304	0.01	304	0.07	304	0.07	304	0.01	233	0.07
root_197.pla	444	0.16	446	0.01	444	0.01	444	0.08	444	0.06	446	0.01	452	0.06
rot_orig.pla	38453	47.58	48344	15.69	78639	61.42	78375	60.18	110887	61.68	7339159	139.15	70456	20.52
ryy6_198.pla	103	0.46	107	0.02	133	0.01	107	0.09	119	0.06	119	0.04	133	0.05
s1196_orig.pla	4104	0.41	4110	0.05	4691	0.19	4691	0.18	4653	0.18	3765	0.17	4688	0.04
sao2_199.pla	657	0.22	698	0.01	667	0.01	653	0.11	684	0.06	657	0.02	716	0.07
sct_orig.pla	405	0.61	513	0.02	545	0.06	545	0.07	548	0.07	387	0.04	539	0.01
seq_201.pla	9349	2.89	9908	0.33	19362	0.43	9216	3.54	15309	0.7	9113	4.17	15364	1.14
sf_232.pla	31	0.07	36	0.02	36	0.01	36	0.05	21	0.05	43	0.01	36	0.06
spla_202.pla	5947	0.66	6092	0.19	5925	0.07	5858	0.88	5925	0.36	5858	0.31	5648	0.36
sqn_203.pla	374	0.13	426	0.02	426	0.01	361	0.08	426	0.06	356	0.01	392	0.06
sqr6_204.pla	470	0.11	524	0.02	486	0.01	486	0.07	486	0.06	482	0.01	486	0.06
sqrt8_205.pla	237	0.16	283	0.01	240	0.01	237	0.07	240	0.06	221	0.01	240	0.06
squar5_206.pla	232	0.09	264	0.02	253	0.01	232	0.07	232	0.06	232	0.01	253	0.07
sym10_207.pla	253	0.24	253	0.03	253	0.01	253	0.1	253	0.09	253	0.02	253	0.1
sym6_63.pla	93	0.11	93	0.01	93	0.01	93	0.06	93	0.06	93	0.01	93	0.05
sym9_71.pla	206	0.2	206	0.03	206	0.01	206	0.07	206	0.06	206	0.01	206	0.06
t481_208.pla	139	0.47	140	0.03	152	0.01	152	0.11	152	0.08	152	0.04	152	0.08
table3_209.pla	5954	0.48	7005	0.08	6276	0.01	5905	0.76	5927	0.13	5905	0.23	6294	0.13
tcon_orig.pla	88	0.51	88	0.02	88	0.05	88	0.06	88	0.06	88	0.01	88	0.01
term1_orig.pla	480	1.76	483	0.06	1193	0.08	1151	0.09	1070	0.08	503	0.26	1154	0.01
tial_214.pla	4475	0.43	5157	0.09	7609	0.03	4284	0.59	4852	0.14	4284	0.18	6952	0.15
too_large_orig.pla	3744	2.93	5379	0.46	5922	0.25	5393	0.25	6334	0.26	2767	2.51	5922	0.08
ttt2_orig.pla	749	0.92	1006	0.04	727	0.07	727	0.07	1135	0.08	734	0.1	722	0.01
unreg_orig.pla	494	1.93	494	0.04	494	0.06	494	0.06	494	0.06	495	0.1	494	0.01
urf4_89.pla	28488	0.77	28833	0.47	28523	0.09	28733	1.83	28523	0.44	28393	0.64	28395	0.45
vda_orig.pla	4224	0.56	4224	0.08	4477	0.14	4477	0.14	4460	0.15	4169	0.19	4470	0.03
wim_220.pla	103	0.07	108	0.01	107	0.01	107	0.05	107	0.05	107	0.01	93	0.05
x1_orig.pla	3567	3.76	3703	0.16	3760	0.13	4277	0.13	4203	0.14	3172	0.9	3760	0.03
x2_223.pla	191	0.23	191	0.03	274	0.01	191	0.08	283	0.06	191	0.01	257	0.05
x2_orig.pla	191	0.2	191	0.02	274	0.05	274	0.06	283	0.07	191	0.01	257	0.01
x3_orig.pla	3941	23.93	4645	0.19	4827	0.26	4815	0.25	4274	0.28	4046	0.7	4929	0.04
x4_orig.pla	2662	11.84	2782	0.11	4459	0.19	4459	0.2	4334	0.2	3006	0.64	4459	0.04
xor5_195.pla	8	0.07	8	0.01	8	0.01	8	0.05	8	0.05	8	0.01	8	0.06
z4_224.pla	66	0.13	66	0.01	66	0.01	66	0.06	66	0.05	66	0.01	39	0.05
z4ml_225.pla	66	0.13	66	0.02	66	0.01	66	0.06	66	0.08	66	0.01	39	0.05

* The letter in parentheses indicates synthesis mode, i.e., (B) means **Balance** mode, (E) means **Efficiency** mode.

†Quantum Cost

‡Synthesis times reported as $0.01s$ represent Revkit's measurement limit ($\leq 0.01s$).